

Sistemas Distribuídos — Semana 03

Comunicação entre Processos e Sincronismo de Relógios · Aulas 5 (18/08) e 6 (20/08) · Prof. Leandro Borges

Comunicação entre processos

Em um sistema distribuído, processos que rodam em máquinas diferentes só conseguem colaborar trocando mensagens pela rede. Essa comunicação envolve várias decisões de projeto que não existem em um programa comum rodando em uma única máquina, e que vamos detalhar nesta aula.

A primeira delas é a transmissão de dados. Como os dados de uma aplicação — objetos, estruturas, variáveis — não podem ser enviados "como estão" pela rede, é preciso convertê-los para um formato serializável antes do envio (marshalling) e reconstruí-los do outro lado (unmarshalling). Formatos como JSON, XML e Protocol Buffers existem justamente para padronizar essa conversão.

A segunda é o endereçamento: como um processo descobre e identifica outro processo na rede? Isso é resolvido por uma combinação de endereços IP, portas, nomes de serviço e, em sistemas mais sofisticados, serviços de descoberta que resolvem nomes lógicos para endereços físicos.

A terceira é o enfileiramento, ou bufferização. Como o emissor e o receptor raramente processam mensagens exatamente na mesma velocidade, é comum usar filas (buffers) para armazenar temporariamente as mensagens que ainda não foram consumidas, evitando que o emissor precise esperar o receptor ficar livre a cada envio.

Confiabilidade na comunicação

Redes não são perfeitas: pacotes podem se perder, chegar fora de ordem ou duplicados. Por isso, sistemas distribuídos usam alguns mecanismos de confiabilidade. A confirmação (ACK) faz o receptor avisar explicitamente que recebeu uma mensagem. Quando essa confirmação não chega dentro de um tempo limite, entra em ação a retransmissão, que reenvia a mensagem automaticamente.

Além disso, é preciso garantir a ordenação — que as mensagens sejam processadas na sequência correta, mesmo que cheguem fora de ordem pela rede — e a detecção de duplicatas, para que uma mesma mensagem retransmitida não seja processada duas vezes.

Vale destacar que protocolos como o TCP já implementam boa parte desses mecanismos na camada de transporte. Ainda assim, muitos sistemas distribuídos precisam de garantias adicionais na camada de aplicação, especialmente quando a operação precisa ser idempotente (poder ser repetida sem efeito colateral) ou transacional.

Por que sincronizar relógios em um sistema distribuído

Depois de resolver como os processos se comunicam, surge um problema mais sutil: como saber a ordem em que eventos aconteceram, se cada máquina tem seu próprio relógio de hardware? Osciladores de hardware não são perfeitos e derivam (drift) em velocidades ligeiramente diferentes ao longo do tempo — então, mesmo relógios ajustados corretamente no mesmo instante vão gradualmente se desalinhar.

Isso importa porque muitas operações em um SD dependem de noção de tempo: ordenar eventos (qual de dois eventos, em máquinas diferentes, ocorreu primeiro?), coordenar ações (por exemplo, dois nós que precisam realizar um backup ou uma transação no mesmo instante) e manter logs e auditorias consistentes entre múltiplas máquinas.

Relógios físicos vs. relógios lógicos

Relógios físicos tentam aproximar o tempo real (UTC) em cada máquina. O algoritmo de Cristian resolve isso fazendo um cliente perguntar a hora a um servidor de tempo confiável e compensando o atraso estimado da rede na resposta. O algoritmo de Berkeley, por sua vez, funciona de forma cooperativa: um coordenador consulta os relógios de todas as máquinas do grupo, calcula uma média e envia a cada uma o ajuste necessário. Na prática, a Internet resolve isso com o NTP (Network Time Protocol), o padrão usado por praticamente todo sistema conectado à rede hoje.

Relógios lógicos seguem uma lógica diferente: eles não tentam medir o tempo real, apenas capturar a ordem causal entre eventos — a relação chamada happens-before. O exemplo clássico é o timestamp de Lamport: cada processo mantém um contador interno, que incrementa a cada evento local e, ao receber uma mensagem, passa a valer o maior entre seu contador atual e o valor recebido, mais um. Essa regra simples garante uma propriedade importante: se um evento A causou (aconteceu antes de, e influenciou) um evento B, então o timestamp de A é sempre menor que o de B — mesmo sem que os relógios físicos das máquinas estejam sincronizados.

Referências

COULOURIS, G. et al. Sistemas Distribuídos: Conceitos e Projeto. 5. ed. Porto Alegre: Bookman.

TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. São Paulo: Pearson.